

Spider And Lobster Procedural Gait Research

Executive Summary

对 `godot_citys` 来说，蜘蛛应当作为第一只正式落地的多足爬行动物：现有研究和开源样例都支持“交替四足 gait + 每条腿独立落脚点搜索 + 身体姿态补偿”的实现路线，而且这种路线对缺腿、失足和不平地形也更鲁棒。[1][2][3][4]

龙虾同样适合程序化实现，但它的生物学步态更接近“后向前传播的 metachronal wave（波式步态）”，同时对身体低姿态、前后向推进和环境介质更敏感；更合理的落地方式不是另起一套系统，而是在同一条 arthropod locomotion spine 上做第二种 profile。[5][6][7]

Key Findings

- **蜘蛛更适合作为首个版本**：蜘蛛步态研究已经给出了比较稳定的低速/中速协调模式，低速下常见交替四足 gait，高速时再进入更动态的模式；这意味着 `v39` 不必从八条腿完全自由振荡开始，而是可以先从两组相位簇做起。[1][2]
- **蜘蛛控制器必须天生允许“失衡但不崩”**：蜘蛛在缺腿后仍能保持稳定，只是效率下降；这直接支持“每条腿局部决策 + 全身轻量补偿”的控制结构，而不是一个脆弱的全局完美解算器。[3][4]
- **龙虾不能直接套蜘蛛 gait**：美洲龙虾的前进步态更像后足到前足传播的 metachronal coordination，而且不同步足承担不同功能；如果直接镜像蜘蛛 gait，视觉上会更像“八脚壳怪”而不是龙虾。[5][6]
- **项目里最值得共享的是 locomotion spine，不是具体 gait**：共享部分应是 `leg state machine + foothold search + body solver + IK + debug contract`；蜘蛛和龙虾差异主要落在 `phase table / duty factor / step height / body clearance / turn bias` 这些 profile 参数上。[4][7][8][9][10][11][12]
- **仓库现实支持 lab-first**：现有 `BuildingCollapseLab`、`HelicopterGunshipLab`、`LakeFishingLab` 已经证明“独立 lab 先调顺，再考虑主世界移植”是本仓库的正式工作流；这次也应完全复用这条路线，而不是直接往主世界塞一只新生物。

Detailed Analysis

1. 蜘蛛步态研究对程序化实现的直接启发

Biomechanics of octopedal locomotion 对捕鸟蛛 *Grammostola mollicoma* 的分析指出，蜘蛛在较低速度下会表现出一种可视作“walk-trot”的协调模式，而在更高速度区间则进入更动态的 running gait；作者还指出，前四足与后四足在动力学上几乎像两个耦合的四足系统。[1]

这对实现非常重要，因为它说明首版不需要一上来就做“8 条腿完全对等、完全自由”的求解器；更现实的做法是：

- 先把 8 条腿拆成左右交错的两大相位簇；
- 再在相位簇内部做微偏移，减少同步抬脚导致的身体抖动；
- 最后才根据速度、转向和失足情况放宽相位约束。

Locomotion and kinematics of arachnids 综述进一步强调，许多蜘蛛在常见移动速度下都依赖交替四足 gait，而不是像昆虫那样的 tripod gait；此外，蜘蛛腿部的液压/弹性特征会影响腿的伸展与回收方式。[2] 对游戏实现而言，这意味着：

- **要学的是“结果约束”而不是完整生理学**：我们不需要模拟真实液压系统，但应当保留“腿在 swing 阶段更像被抛出并落地、而不是机械铰链匀速插值”的运动感觉。
- **步态重心不在髋关节动画，而在脚端 contract**：只要脚端时序、身体高度、俯仰/横滚补偿做对，视觉上就已经很像蜘蛛。

Limping following limb loss increases locomotor stability but reduces locomotor efficiency in spiders 的结论尤其适合游戏工程：缺腿会降低效率，但不会让蜘蛛立刻失稳崩坏，说明蜘蛛 locomotion 本来就是冗余且分布式的。[3]

对 godot_citys 的直接启发是：

- 每条腿都应维护自己的 stance / lift / swing / plant 局部状态；
- 如果某条腿找不到可落脚点，不要全体重算，而是让它延迟抬脚、缩短步长或直接跳过一个 cycle；
- 身体姿态求解器只做“尽量满足”，不追求每帧完美闭式解。

ETH 的硕士论文 Locomotion of Spiders - What Robotics can Learn from Spiders and Vice Versa 则把蜘蛛的多腿冗余、黏附/地形适应和机器人实现问题连在一起，给出的总方向也是“局部接触决策 + 分布式协调 + 简化模型优先”。[4]

因此，对本项目最务实的路线不是“先买一个超复杂蜘蛛动画资产”，而是：

1. 先用 blockout/proxy rig 把腿端 contract 跑通；
2. 再换正式蜘蛛视觉资源；
3. 最后根据正式模型调整 socket、长度和限位。

2. 龙虾步态研究的实现含义

Macmillan 对 `Homarus americanus` 的经典分析表明，美洲龙虾前进时最常见的是一种后向前传播的 metachronal gait，而且不同步足在一个 gait cycle 里承担不同功能，例如后侧步足更偏拉动、中央步足更偏横向划动、前侧步足更偏推动。[5]

这直接否定了“把蜘蛛控制器换个壳就是龙虾”的做法。龙虾 profile 至少应在以下方面独立设置：

- **phase ordering**：按后向前的波推进，而不是蜘蛛式交错 tetrapod；
- **step arc**：抬脚高度更低、摆动更短、更贴地；
- **body clearance**：身体更低、更贴地面；
- **turn bias**：转向时更依赖身体 yaw 和相邻腿步幅差，而不是明显的侧向跨步。

`Skeletal adaptations for forwards and sideways walking in three species of decapod crustaceans` 给出一个很有用的分类视角：前行型 decapod 和侧行型 decapod 在骨架布局上就不同，前行型更适合把推进方向与身体朝向保持一致。[6]

如果我们要做“龙虾而不是螃蟹”，就应当明确冻结：

- `v39` / 首版龙虾是 **forward crawler**，不是 sideways crab；
- 身体和头胸甲朝向应基本对齐速度方向；
- 螯足可以参与视觉姿态和轻微支撑，但不应在首版里承担主要推进责任。

`Underwater walking` 的综述进一步指出，十足类步行并不是单纯由一个中央节拍器完成，而是高度依赖局部感知、负载反馈和相邻腿协调。[7]

这对游戏实现的意义是：龙虾也不适合用“固定循环动画 + 脚底吸附”的假法硬装成程序化。更稳妥的方式仍然是共享一条 per-leg controller，只是把 gait table 换成波式 profile。

3. 对 `godot_citys` 的实现扩展建议

结合上面的研究与仓库现状，我更建议把本项目里的 arthropod locomotion 切成四层：

Layer 1: Shared locomotion spine

这是蜘蛛和龙虾都共享的正式 runtime，建议职责冻结为：

- `ArthropodLocomotionProfile`
 - 腿数量
 - socket/bone 路径
 - 默认相位表
 - `duty_factor`
 - `step_length`
 - `step_height`
 - `body_clearance`

- 转向权重
- 地形法线跟随权重
- **ArthropodLegRuntime**
 - **stance**
 - **lift**
 - **swing**
 - **plant**
 - 当前锁定脚点
 - 上一次成功落脚点
 - 伸展阈值 / 超时阈值
- **ArthropodBodySolver**
 - 依据所有 **stance** 脚点估计支撑面
 - 估算身体目标高度、roll、pitch
 - 对转向和加速度做轻量前馈补偿
- **ArthropodFootholdResolver**
 - 以理想脚点为中心做若干射线/扇形采样
 - 输出“可落脚点 + 法线 + 命中层信息”
- **ArthropodDebugState**
 - gait phase
 - 每条腿的 mode
 - 目标脚点 / 实际脚点
 - 机体姿态误差
 - 未命中计数 / 重新规划计数

这层一旦成立，蜘蛛和龙虾就只是两套 profile，而不是两个系统。

Layer 2: Species wrapper

这一层做“物种差异注入”，而不是重复造轮子：

- **SpiderCrawlerRuntime**
 - 8 条腿
 - 交替四足 gait
 - 较高 step height
 - 更强地形跨越
 - 允许更大的身体姿态变化
- **LobsterCrawlerRuntime**
 - 首版以前四对步足为 walking set
 - metachronal wave
 - 更低 clearance

- 更短步长
- 更稳定的 body yaw / roll

Layer 3: Lab wrapper scene

这层只负责：

- 玩家 / 观察相机
- 地面与障碍 authoring
- reset / debug HUD
- 生物实例挂接
- 单独的 F5 reset / F6 运行体验

它不应该拥有独立 gait 逻辑。现有

`BuildingCollapseLab.gd`、`HelicopterGunshipLab.gd`、`LakeFishingLab.gd` 都已经体现了这种职责分离。

Layer 4: Main-world portability hooks

虽然本轮不接入主世界，但现在就要冻结可移植口径：

- 主世界 future consumer 不应直接 new 一个 lab scene；
- 应该由 shared runtime + world wrapper 进行挂接；
- 应预留：
 - world anchor
 - ground resolver
 - chunk-aware activation
 - encounter / ambient / task integration hook
 - future full-map / task pin / feature registry hook

这和 v37 炮艇要求“lab runtime == main-world runtime”是同一设计哲学。

4. 在本仓库里先做蜘蛛、再做龙虾，需要补哪些东西

4.1 先做蜘蛛时必须具备的东西

1. 独立 spider lab 场景

- 形式上对齐现有 `city_game/scenes/labs/*`。
- 包含平地、坡面、低台阶、窄梁四种地形工况。

2. 共享 locomotion spine

- 首版不要写死成蜘蛛专用脚本。

3. 每条腿的落脚点 contract

- stance 时锁脚；

- 只有超过伸展阈值或地形丢失时才重规划；
- 一次只允许有限数量的腿同时 swing。

4. 轻量 IK 解算

- 若正式模型是短链腿，优先自写解析/半解析 solver；
- 若 rig 更复杂，可引入 `GodotIK` 或 `ISOK` 做 authoring/调试辅助。[8][9]

5. 调试视图

- 必须可视化理想脚点、实际脚点、stance polygon、当前 gait phase。

6. 测试

- scene contract
- gait contract
- terrain follow contract
- missing foothold recovery contract
- reset contract

4.2 再做龙虾时必须新增/修改的东西

1. 不要新建第二套 runtime

- 只新增 `LobsterLocomotionProfile` 和 species wrapper。

2. 更换 gait scheduler

- 由蜘蛛的 tetrapod 改成 metachronal wave。

3. 改低 body clearance 与步高

- 龙头胸甲和尾节更接近地面。

4. 定义螯足 contract

- 首版可只做姿态摆动与威吓，不承担主推进。

5. 决定环境口径

- 更生物真实的做法是浅水/湿滑平台 lab；
- 若直接在干地做，也要在文档里承认这是 stylized lobster。

5. 开源项目参考价值

我找到的开源参考里，真正“拿来就能抄”的 lobster 项目明显少于 spider / hexapod；更现实的参考组合是“Godot IK 工具 + spider 实战样例 + generic multi-legged robot sandbox”。

- **GodotIK**：Godot 的开源 IK 插件，适合做 rig authoring、骨骼目标调试和 solver 对比，不一定要原样进入 runtime 热路径。[8]
- **ISOK**：面向 Godot 4.5 的轻量 IK 插件，适合快速试 rig 和看目标骨链是否稳定。[9]
- **Unity-Procedural-IK-Wall-Walking-Spider**：虽然是 Unity 项目，但它把“腿端锁定 + 程序化摆腿 + wall-walking spider”这条思路演示得很直接，对蜘蛛 lab 的视觉节奏很有参考价值。[10]
- **HexapodRobotSimulation**：更偏机器人仿真，但对 gait phase、足端规划、地形验证很有价值，适合作为第二参考面。[11]

- **WalkingSpider_OpenAI_PyBullet_ROS**：偏研究/仿真环境，不适合直接搬到游戏里，但对“多腿体的参数化控制与训练接口”有启发。[12]

基于这些来源，我的判断是：

- 蜘蛛有现成实现思路可借；
- 龙虾缺少成熟的开源游戏式样例；
- 因此，龙虾更适合建立在蜘蛛已经跑通的 shared spine 上，而不是和蜘蛛并行起两套系统。

6. 与 `godot_citys` 现状结合后的版本建议

结合仓库现状：

- 现有独立 lab 工作流已经成熟；
- `city_game/assets/environment/source/creatures/` 下当前有 `lobster_02.glb`，但没有现成蜘蛛模型；
- 因此版本规划最合理的冻结方式是：

1. `v39` 做 **arthropod crawler labs foundation**

- shared locomotion spine
- spider lab first
- lobster lab second
- 只做 main-world portability hooks，不做正式主世界接入

2. 蜘蛛首版允许 **proxy/blockout rig**

- 先把 gait contract 跑通
- 再替换正式美术

3. 龙虾直接复用现有 `lobster_02.glb`

- 但 locomotion 仍由 shared runtime 驱动

Areas of Consensus

- **多足生物适合程序化脚端控制**：无论是蜘蛛研究还是十足类/机器人研究，都更支持“局部接触决策 + 相位协调 + 身体补偿”，而不是纯 baked clip。[1][2][3][4][5][7]
- **蜘蛛和龙虾可以共享一条 locomotion spine**：共享的是足端控制、相位调度、姿态求解与调试 contract；不共享的是 gait table 和 species profile。[4][5][6][7]
- **蜘蛛应先于龙虾实现**：研究资料、开源参考和生物学步态抽象都更有利于蜘蛛率先成为 `v39` 第一只正式生物。[1][2][3][8][9][10]

Areas of Debate

- **v39 是否应支持蜘蛛爬墙/倒挂**：开源蜘蛛样例很强调 wall-walking，但这会显著抬高地形法线和重力框架复杂度；对仓库首版来说，先做地面/坡面/障碍 crawling 更稳妥。[10]
- **龙虾是否必须放到浅水环境**：从生物真实性看，浅水/湿平台更合理；从工程节奏看，干地 stylized lab 更便于先验证 gait spine。[5][6][7]
- **IK 是否应依赖通用插件**：GodotIK / ISOK 很适合 authoring 与验证，但正式 runtime 未必需要重依赖；腿链短、结构固定时，自写 solver 可能更可控。[8][9]

Sources

[1] Biancardi, C. M., Fabrica, C. G., Polero, P., Loss, J. F., & Minetti, A. E. "Biomechanics of octopedal locomotion: kinematic and kinetic analysis of the spider *Grammostola mollicoma*." *Journal of Experimental Biology* (2011). 链接：<https://journals.biologists.com/jeb/article/214/20/3433/10466/Biomechanics-of-octopedal-locomotion-kinematic-and> （同行评审实验研究，高可信）

[2] Wolff, J. O. "Locomotion and kinematics of arachnids." *Current Opinion in Insect Science* / PMC open-access version (2021). 链接：<https://pmc.ncbi.nlm.nih.gov/articles/PMC8046687/> （综述，高可信）

[3] Wilson, R. S., et al. "Limping following limb loss increases locomotor stability but reduces locomotor efficiency in spiders." *Journal of Experimental Biology* (2018). 链接：<https://journals.biologists.com/jeb/article/221/21/jeb185991/27695/Limping-following-limb-loss-increases-locomotor> （同行评审实验研究，高可信）

[4] Göttler, G. "Locomotion of Spiders - What Robotics can Learn from Spiders and Vice Versa." ETH Zurich Master Thesis (2021). 链接：<https://www.research-collection.ethz.ch/handle/20.500.11850/502395> （高校正式论文，面向机器人实现，可信）

[5] Macmillan, D. L. "A physiological analysis of walking in the American lobster (*Homarus americanus*)." *Journal of Comparative Physiology* (1975). 链接：<https://pubmed.ncbi.nlm.nih.gov/234622/> （经典实验论文摘要，可信）

[6] Vidal-Gadea, A. G., Minter, R., et al. "Skeletal adaptations for forwards and sideways walking in three species of decapod crustaceans." *Journal of Experimental Biology* (2008). 链接：<https://journals.biologists.com/jeb/article/211/16/2613/18201/Skeletal-adaptations-for-forwards-and-sideways> （同行评审实验研究，高可信）

[7] Ayers, J., et al. "Underwater walking." *Current Biology* / *Trends in Neurosciences style overview* (abstract/overview entry). 链接：

<https://www.sciencedirect.com/science/article/abs/pii/S1467803907000370> (综述型来源, 作为控制结构背景, 可信中高)

[8] monxa. “GodotIK.” GitHub repository. 链接: <https://github.com/monxa/GodotIK> (官方仓库, 开源实现参考)

[9] okaysalmon. “ISOK.” GitHub repository. 链接: <https://github.com/okaysalmon/ISOK> (官方仓库, Godot IK 工具参考)

[10] PhilS94. “Unity-Procedural-IK-Wall-Walking-Spider.” GitHub repository. 链接: <https://github.com/PhilS94/Unity-Procedural-IK-Wall-Walking-Spider> (官方仓库, 蜘蛛程序化步态/贴附参考)

[11] XuelongSun. “HexapodRobotSimulation.” GitHub repository. 链接: <https://github.com/XuelongSun/HexapodRobotSimulation> (官方仓库, 多足 gait 仿真参考)

[12] rubencg195. “WalkingSpider_OpenAI_PyBullet_ROS.” GitHub repository. 链接: https://github.com/rubencg195/WalkingSpider_OpenAI_PyBullet_ROS (官方仓库, 蜘蛛机器人控制/仿真参考)

Gaps and Further Research

- **蜘蛛正式视觉资产仍缺失**: 仓库当前没有现成蜘蛛模型; v39 应允许 proxy rig 先行, 后续再替换正式资产。
- **龙虾场景环境语义尚未冻结**: 要先明确它是“浅水/湿平台真实风格”还是“干地 stylized crawler”。
- **主世界消费者还未定义**: 未来它可以是 scene landmark、world feature、ambient creature, 还是 task/event encounter, 这会反过来影响 wrapper 设计。
- **性能边界需要尽早量化**: 8 条腿、每腿多射线、每帧 IK 和身体补偿在真实渲染下的代价需要专门 profiling, 不能只看 headless。